

ML PROGRAMMING – LINEAR REGRESSION CHECK LIST

Preparation – The Foundation

1. Prepare a Statement of Intent: *What – Why – How*

What: We are going to build a ML solution for analyzing Oil Production from Fracking Oil Wells. The target is labeled *Log_Cum_Production*, which is the log of the first years production for a well. The data are several thousand oil wells in Texas. We want to forecast oil production using the well characteristics, such as operator.

Why: Forecasting oil production for a new well is important for several reasons. First government officials need to plan on revenue from new wells in order to anticipate economic benefits from the production. Second, investors need to know what to expect from their investment in building a new well.

How: You are my assistant software engineer experienced at programming python machine learning software. We are constructing this model together, step by step. Please do not get ahead of me. Wait for instructions after completing each step. I will run and test all code using my python IDE, Spyder with the *environment* **only656**. The *AdvancedAnalytics* package used in this course is installed in that environment.

2. Create a *Project Folder* with 4 *Sub-Folders*

You can create folders inside Cursor, or the OS on your computer. Here are the 4 sub-folders that are always useful for organizing your files. Use a folder name that helps you easily identify the Project, e.g. “*CapstoneDataCleaning*”, or “*CapstoneRegAnalysis*”

FOLDER	PURPOSE	DESCRIPTION – POSSIBLE SUB-FOLDERS
Templates	Storing coding examples used for training Agents on how you like to implement your ML solutions.	These are organized into sub-folders that contain a collection of examples for a particular ML technique. Examples would be AdvancedAnalytics for the package, Linear Reg. for ML analysis of Interval targets using regression, Logistic Reg. For analysis of binary and nominal targets with logistic regression.
Data	Storing data files used by programs in the Templates folder.	Most of these files will be excel and csv files since most ML algorithms focus on forecasting targets that are either interval, binary or nominal. However, beyond ML, we get involved in the analysis of images and text. These data would also be stored in this folder, but using other sub-folder, such as “Customer Reviews”. It is useful to organize data files into sub-folders, usually organized by the data category.
Tools	Storing Tools used to support your project	These tools are those you might use to prepare the data for analysis or extract data from other sources. This includes the data_map_tool.py used to create an initial code example containing a data map dictionary for your data.
Notes	Storing prompts communicated to your Agents and example plots and program output.	Typically this would be your favorite prompts for coaching your Cursor Agent. These might be organized by the ML algorithm category, or data type.

In addition to these sub-folders, the **project folder** contains the code you are developing as well as *instructions, example output, images/plots and data dictionaries*.

3. Prepare Data Dictionary

All ML projects require data described by a data map. This describes the names of every attribute in the project data, its data type and allowed values. The names need to be acceptable as a valid python name, the data type is Interval, Binary, Nominal or Text and the allowed values for each attribute depends on the nature of each attribute. Ideally these come from project specification prepared independently of the data.

Data_Map_Tool.py This program can be used to draft a data_map from a data file. The data_map is stored in a python file. The python file is stored in a .py file that you name. Normally this is your initial file for your project. Most likely you will need to edit the data map in this file because sometimes the program labels an attribute as Nominal instead of Interval, and vice versa. Typically the file is edited to correct data types and possibly data limits.

Implement – Step by Step after Preparing the Foundation

Step 1 – Train the Agent Ask your agent to review the example solutions found in Templates that are related to your project. Also review the **AdvancedAnalytics** package in the Templates folder. Reinforce your “how” prompt, how you would like the Agent to work on each step, one at a time. Otherwise it is not unusual for an Agent to review the templates and then start writing code based upon the data it sees in the **Data** sub-folder.

Step 2 - Data Tell your Agent where the project data is located. Say “**data are in the file <data filename>**”. If you created a proper data map, let the agent know the name of the file you are using. The one that contains the data map and possibly a few statements for reading the data. If you don’t have a data map file, ask the agent to create a data map in a program labeled **<program filename>**. If this is a new file, review it looking for incorrect data types or data limits. We are using the data types described in **AdvancedAnalytics.ReplaceImputeEncode**. That method will clean and prepare our data for regression analysis. Replace all outliers with missing values, and estimate missing values for all attributes except the target. Give the Agent the name of the targets in your data. Ensure the Agent understands the data types for your targets, as well as other attributes you plan to use in the analysis.

Step 3 – Hyperparameter Optimization Hyperparameter optimization depends on the category of the ML model being used. For regression, hyperparameter optimization involves selecting the best attributes to include in the model. The optimization could also involve decision regarding possible scaling of these attributes and transformation, such as using the log of some data attributes. In Regression, there are three approaches to hyperparameter optimization: *Stepwise*, *Lasso* and *All-Possible Subsets*. In some cases more than one of these approaches might be used.

If a dataset contains many data attributes, the All-Possible Subsets approach is not practical. This search involves trying every possible combination of features. If you have k candidate features in your data then an All-Possible Subsets search would evaluate 2^k models. If k=10, then the number of possible candidate models is 1,024. That might be possible, but if k=20, then you have over a million candidates.

For very large datasets, data with millions of observations and hundreds of candidate features, even stepwise selection might not be practical. In these cases, the best approach is to use Lasso

regularization. You can use special algorithms to find the optimum regularization parameter, then you fit a model using that parameter and identify candidate features by those with non-zero coefficients. These approaches are specifically designed for regression models.

Other ML algorithms use different approaches and involve other issues beyond feature selection.

Step 4 – Validation The objective of validation is to evaluate how well the model will do if used to forecast or classify future data. For that we fit the model using one set of data. The model is then asked to provide forecasts or classifications for another dataset, one that was not used to build the initial model.

There are two popular approaches to validation: Hold-Out validation and Cross-Validation. A 70/30 hold-out validation approach is favored for large datasets or ones with many attributes. This approach randomly selects 70% of the data to construct the model and then validates it by having it forecast the 30% that was not used to build the model. This is fairly straightforward although sometimes stratification is important. This is particularly important when the target is binary or nominal and when some of the key features are binary or nominal.

The other form of validation, cross-validation, expands upon the basic hold-out validation. Instead of just doing one 70/30 hold-out analysis, the data are randomly divided into several training/hold-out samples. Each one is used to build the model and then each is independently evaluated comparing the forecasts or classification between the training and hold-out samples.

If the dataset is not too large, then the cross-validation approach provides a better evaluation of the ML solution. In general the cross-validation approach is preferred, but only when the size of the dataset permits this analysis.

For both hold-out and cross-validation, the average squared error (ASE) is used to validate a forecasting model. One in which the target is Interval. Over fitting is a possibility for poor models. Overfitting occurs when the ASE for the training data is much better than the ASE for the hold-out sample. This is quantified by looking at the ratio of the ASE for the hold-out sample to the training sample. That is, $\text{ratio} = \text{ASE_Validation} / \text{ASE_Training}$. If this is greater than 1.2, it is common to say that the model is overfitting the data. The model is fitting the training data much better than it fits the hold-out sample.

For ML analysis involving classifications, ASE is replaced with “Accuracy” or “MISC” misclassification. Accuracy is the percent of times the classification is correct, and MISC is the percent of incorrect classifications. This calculation does vary depending upon whether the classification is Binary or Nominal. We still use the ratio of the training and validation data as a rule of thumb for identifying overfitting of a classification model.

If misclassification is used, $\text{ratio} = \text{MISC_Validation} / \text{MISC_Training}$, and again the overfitting trigger is a ratio greater than 1.2. If accuracy is used instead of misclassification, then the ratio is inverted, $\text{ratio} = \text{Accuracy_Training} / \text{Accuracy_Validation}$.

Misclassification and accuracy are easily defined for binary classification, but for nominal there are several ways to calculate accuracy and misclassification.

Step 5 – Diagnostics The health of the final model is evaluated using an analysis of standardized residuals. A fast diagnosis is to calculate the number of standardized residuals beyond 2, 3 & 6 standard deviations, and then compare those numbers to what is expected. In most cases the number expected beyond ± 6 is zero. In that case, any case with a standardized residual below

-6 or above +6 would be considered an outlier. It could be a data artifact or a problem with the model.

Another tool for evaluating the quality of a ML model is to use diagnostic plots of the standardized residuals. The most common plots are: 1) a *histogram of the Std_Residuals*, 2) the *Std. Residuals vs data order*, 3) the *Std_Residuals vs Predicted Values*, 4) the *Std_Residuals vs Cooks-d*.

Step 6 – Save Store the *program output* in a text or ‘md’ file, and the *diagnostic plots* in png or jpg files. Normally these are saved in the *Project Folder* alongside the ML program used for the analysis.

Step 7 – Memory Ask your Agent to store the *context* of the analysis in the *Notes sub-folder* in a markdown file for future review. If you continue the same chat, Cursor still has that conversation. Cursor may also remember a few facts about you. A new chat does not receive the analysis. At the start of a new session, point the agent to your program and to the markdown file you stored in the *Notes sub-folder*. Saving this file is good protection for your work.

Step 8 – Independent Review Unfortunately, none of the Cursor Frontier models are available in “free” Cursor accounts. An independent review of your ML analysis requires a Frontier model. Today the best are either Fable 5.1 or Opus 5. Opus 5 is less expensive than Fable 5.1, but neither are expensive for this limited use. That is, if you restrict them to evaluating the analysis produced from your project program. They are willing and eager to go beyond this inspection, but avoid that if you are concerned about additional charges to your account. Cursor will provide warnings when this is happening, which is reassuring. The documentation for Cursor goes into this in some detail, and if you are concerned or confused, just ask your Cursor Agent for clarification.

How to get an independent review? First, if you haven’t installed the *yzane extension* in Cursor for converting *md* to *pdf*, then do so. It’s available at no extra charge. Again, your Cursor Agent can guide you to installing *yzane*; it’s easy, and once it’s installed you can read and write pdf, which is very useful.

Next, start a *new Cursor agent* in your Cursor project, and change its model to a Frontier *reasoning* model *Fable 5.1* or *Opus 5*. Ask your new Agent to review the output from your analysis. You saved that in *Step 6* above. Ask your reasoning Agent to write an opinion report, including recommendations. Label the opinion report *{model}_opinion.pdf*. Ask your Agent to include its interpretation of the analysis, possible shortcomings and limitations of your analysis and its recommendations for improvements.